

BÀI TẬP TÌNH HUỐNG – ÁP DỤNG STRATEGY PATTERN

Chủ đề: Hệ thống đặt sân thể thao trực tuyến

Bối cảnh

Một công ty phát triển hệ thống đặt sân thể thao online cho:

- sân bóng đá
- sân tennis
- sân cầu lông
- sân pickleball

Hệ thống cho phép khách hàng:

- đặt sân theo giờ
 - thanh toán online
 - áp dụng khuyến mãi
 - tính giá theo nhiều cách khác nhau
-

Quy tắc tính giá hiện tại

Mỗi loại sân có cách tính giá khác nhau:

Loại sân	Quy tắc tính giá
Sân bóng đá	Giá theo giờ cố định
Sân tennis	Giá thay đổi theo khung giờ
Sân cầu lông	Giá cuối tuần cao hơn
Sân pickleball	Giá theo số người chơi

Ví dụ:

Football:
300.000 VNĐ / giờ

Tennis:
6h-18h → 200.000
18h-22h → 350.000

Badminton:

Weekday → 100.000

Weekend → 150.000

Pickleball:

80.000 / người

Thiết kế ban đầu

Lập trình viên viết:

```
public class BookingService
{
    public double CalculatePrice(
        string courtType,
        int hours,
        int players)
    {
        if(courtType == "Football")
        {
            return 300000 * hours;
        }
        else if(courtType == "Tennis")
        {
            return 350000 * hours;
        }
        else if(courtType == "Badminton")
        {
            return 150000 * hours;
        }
        else if(courtType == "Pickleball")
        {
            return 80000 * players;
        }

        return 0;
    }
}
```

Vấn đề phát sinh

Sau một thời gian:

- xuất hiện nhiều loại sân mới
- nhiều chương trình giá khác nhau
- giá thay đổi theo:
 - giờ vàng
 - cuối tuần
 - thành viên VIP
 - thời tiết
 - giải đấu

Code trở nên:

- rất nhiều `if-else`
 - khó bảo trì
 - khó test
 - vi phạm Open/Closed Principle
-

Yêu cầu của trưởng nhóm

Thiết kế lại bằng:

Strategy Pattern

Mục tiêu:

- mỗi thuật toán tính giá là một strategy riêng
 - có thể thay đổi cách tính giá runtime
 - dễ mở rộng loại sân mới
-

YÊU CẦU SINH VIÊN

Phần 1 — Phân tích bài toán

Câu 1

Phân tích các vấn đề của thiết kế hiện tại.

Câu 2

Vì sao bài toán này phù hợp với Strategy Pattern?

Câu 3

Xác định:

- Strategy
 - Concrete Strategy
 - Context
-

Phần 2 — Thiết kế UML

Sinh viên hãy vẽ:

- Class Diagram
- Quan hệ giữa các strategy

Gợi ý:

```
PriceStrategy
    ↑
FootballPriceStrategy
TennisPriceStrategy
BadmintonPriceStrategy
PickleballPriceStrategy
```

Phần 3 — Lập trình bằng C#

Bước 1 — Tạo Strategy Interface

```
public interface IPriceStrategy
{
    double CalculatePrice(
        int hours,
        int players);
}
```

```
}
```

Bước 2 — Tạo Concrete Strategy

Football Strategy

```
public class FootballPriceStrategy : IPriceStrategy
{
    public double CalculatePrice(
        int hours,
        int players)
    {
        return 300000 * hours;
    }
}
```

Tennis Strategy

```
public class TennisPriceStrategy : IPriceStrategy
{
    public double CalculatePrice(
        int hours,
        int players)
    {
        return 350000 * hours;
    }
}
```

Badminton Strategy

```
public class BadmintonPriceStrategy : IPriceStrategy
{
    public double CalculatePrice(
        int hours,
        int players)
    {
        return 150000 * hours;
    }
}
```

Pickleball Strategy

```
public class PickleballPriceStrategy : IPriceStrategy
{
    public double CalculatePrice(
        int hours,
        int players)
    {
        return 80000 * players;
    }
}
```

Bước 3 — Tạo Context

```
public class BookingContext
{
    private IPriceStrategy strategy;

    public BookingContext(IPriceStrategy strategy)
    {
        this.strategy = strategy;
    }

    public double GetPrice(
        int hours,
        int players)
    {
        return strategy.CalculatePrice(
            hours,
            players);
    }
}
```

Bước 4 — Chạy thử

```
class Program
{
    static void Main()
    {
        BookingContext footballBooking =
```

```
        new BookingContext(
            new FootballPriceStrategy());

Console.WriteLine(
    footballBooking.GetPrice(2, 0));

BookingContext pickleballBooking =
    new BookingContext(
        new PickleballPriceStrategy());

Console.WriteLine(
    pickleballBooking.GetPrice(0, 4));
    }
}
```

KẾT QUẢ MONG ĐỢI

600000
320000

Phần 4 — Yêu cầu mở rộng

Yêu cầu 1 — Happy Hour Pricing

Khung giờ:

- 12h–17h giảm 20%

Thiết kế strategy mới.

Yêu cầu 2 — Weekend Pricing

Cuối tuần:

- tăng 30%

Không sửa strategy cũ.

Yêu cầu 3 — VIP Member Pricing

Khách VIP:

- giảm 15%

Yêu cầu 4 — Tournament Pricing

Nếu đặt sân cho giải đấu:

- giá riêng
- phụ phí trọng tài

Yêu cầu 5 — Dynamic Strategy Runtime

Cho phép người dùng chọn:

1. Football
2. Tennis
3. Badminton
4. Pickleball

runtime sẽ đổi strategy tương ứng.

Phần 5 — Câu hỏi tư duy

Câu 1

Nếu không dùng Strategy:

- hệ thống sẽ gặp vấn đề gì khi có 20 cách tính giá?
-

Câu 2

Strategy giúp tuân thủ nguyên lý SOLID nào?

Câu 3

Điểm khác nhau giữa:

- Strategy Pattern
 - State Pattern
-

Câu 4

Khi nào nên dùng:

- Strategy
 - if-else đơn giản
-

Câu 5

Strategy có thể kết hợp với Factory Pattern không?

Nếu có:

- vì sao?
-

BÀI TOÁN MỞ RỘNG THỰC TẾ

Ví dụ 1 — Tính phí Grab/Uber

Các strategy:

- BikePricing
- CarPricing

- SurgePricing
 - AirportPricing
-

Ví dụ 2 — Thanh toán E-Commerce

Các strategy:

- CreditCardPayment
 - EWalletPayment
 - CODPayment
-

Ví dụ 3 — Shipping Cost

Các strategy:

- StandardShipping
 - ExpressShipping
 - InternationalShipping
-

MỨC ĐỘ NÂNG CAO (CHO SINH VIÊN KHÁ/GIỎI)

Bài toán nâng cao — AI Dynamic Pricing

Giá sản thay đổi theo:

- thời tiết
- số lượng booking
- giờ cao điểm
- AI demand prediction

Sinh viên hãy:

- áp dụng Strategy
 - cho phép thay đổi pricing algorithm runtime
-

KẾT QUẢ HỌC TẬP MONG ĐỢI

Sau bài này sinh viên có thể:

- Hiểu bản chất Strategy Pattern
- Thay thế if-else bằng polymorphism
- Thiết kế hệ thống pricing linh hoạt
- Áp dụng SOLID trong thực tế
- Hiểu runtime behavior switching

GỢI Ý THẢO LUẬN TRÊN LỚP

- “Nếu thêm AI Pricing thì sửa ở đâu?”
- “Nếu nhiều strategy dùng chung logic thì xử lý thế nào?”
- “Strategy có gây nhiều class quá không?”
- “Có nên dùng enum + switch thay vì Strategy?”
- “Strategy khác State ở điểm nào?”
- “Strategy có nên inject bằng Dependency Injection không?”